CAMPUS COMPLAINT MANAGEMENT SYSTEM

Team Members:

Pinto N. Sebastian
Gowtham Krishna S
Leo Philip Joseph
Stini Sojan
Jhanvi Rajesh

JUNE 2026

Table of Contents

Table of Figures

Abstract

The Campus Complaint Management System (CCMS) is a centralized, web-based platform engineered to streamline the grievance redressal process within educational institutions. Traditionally, managing student feedback and infrastructure complaints involved manual paperwork or decentralized emails, leading to severe bottlenecks, lost data, and a profound lack of transparency. The CCMS resolves these systemic challenges by digitizing the entire workflow utilizing the MERN full-stack architecture (MongoDB, Express.js, React.js, and Node.js). The application introduces role-based access control, securely isolating the interfaces for Students and Administrators via JSON Web Token (JWT) authentication. Students are empowered with a comprehensive dashboard where they can submit grievances related to various campus facilities, track the real-time resolution status of their issues, and edit pending entries. Simultaneously, administrators are equipped with an advanced filtering console, allowing them to sort complaints by category or date and transition case statuses efficiently. This project simulates a three-week Agile development sprint, demonstrating the integration of high-performance frontend interfaces with robust backend logic, ultimately delivering a transparent, accountable, and highly responsive infrastructure management tool.

1. Introduction

In modern educational institutions, managing student feedback, complaints, and requests manually poses significant administrative challenges. Traditional systems rely on paper forms, suggestion boxes, or unorganized emails, which suffer from systemic flaws. Grievances are frequently misplaced, processing times are slow and unpredictable, and students have no visibility into the current status of their concerns. Furthermore, administrative personnel are forced to manually sort and direct complaints to various departments, making it difficult to prioritize critical issues.

Digitizing this workflow is essential to establish trust, transparency, and operational efficiency. The Campus Complaint Management System (CCMS) is a MERN Full Stack Project motivated by the need to replace outdated, manual grievance processes with an integrated, secure, and user-centric web platform. The objective of this project is to implement role-based authentication, provide intuitive dashboards for both students and administrators, and enforce strict state safety rules protecting data integrity. This system ultimately showcases the power of full-stack development in solving real-world administrative bottlenecks.

2. Problem Statement

The manual administration of student complaints in large campus environments is plagued by several core issues that this Full Stack Project seeks to tackle:

- Lack of Transparency: Students file grievances without receiving any feedback loop, remaining unaware if their concern is received, pending, in-progress, or resolved.
- Inefficient Tracking & Routing: Administrative staff struggle to keep records of complaints, leading to misplaced files and delayed resolutions.
- Lack of Accountability: Without structured logs, it is difficult to determine which department is responsible for a delay or whether service-level objectives are being met.
- Data Redundancy and Security Flaws: Storing sensitive student credentials and complaint data in unsecured physical sheets makes information vulnerable to leaks and unauthorized manipulation.

The CCMS project transforms this chaotic environment into a streamlined digital process, providing real-time data flow between the user interface and secure database records.

3. User Stories

User Story 1: Student Complaint Submission

- As a Student,

- I want to submit a detailed complaint specifying category and location,

- So that campus administration is alerted to the issue immediately.

Acceptance Criteria: The form must validate that no fields are empty. Upon submission, the complaint must default to a 'Pending' status and appear instantly in the student's personal complaint ledger.

User Story 2: Student Status Tracking

- As a Student,

- I want to view my dashboard and see the real-time status of my submitted complaints,

- So that I know whether action is being taken without having to manually follow up.

Acceptance Criteria: The student dashboard must display accurate tallies of total, pending, and resolved complaints, pulling updated statuses from the database.

User Story 3: Admin Status Management

- As an Administrator,

- I want to update the status of a specific complaint (e.g. to 'In Progress' or 'Resolved'),

- So that students are informed of the administrative resolution progress.

Acceptance Criteria: The admin can only update the status field. They cannot edit the student's original description. Changing the status automatically locks the student out from editing or deleting the complaint.

User Story 4: Secure Role Isolation

- As a System Security Manager,

- I want to ensure that students cannot access the admin control panel,

- So that unauthorized users cannot alter global complaint statuses.

Acceptance Criteria: Accessing admin routes requires a valid JWT token carrying an 'Admin' role flag. Invalid access attempts must return a 403 Forbidden error.

4. Flow Chart

The system logic flows according to the following sequenced steps based on the user stories. The flowchart below provides a visual representation of how the system works based on the user stories.

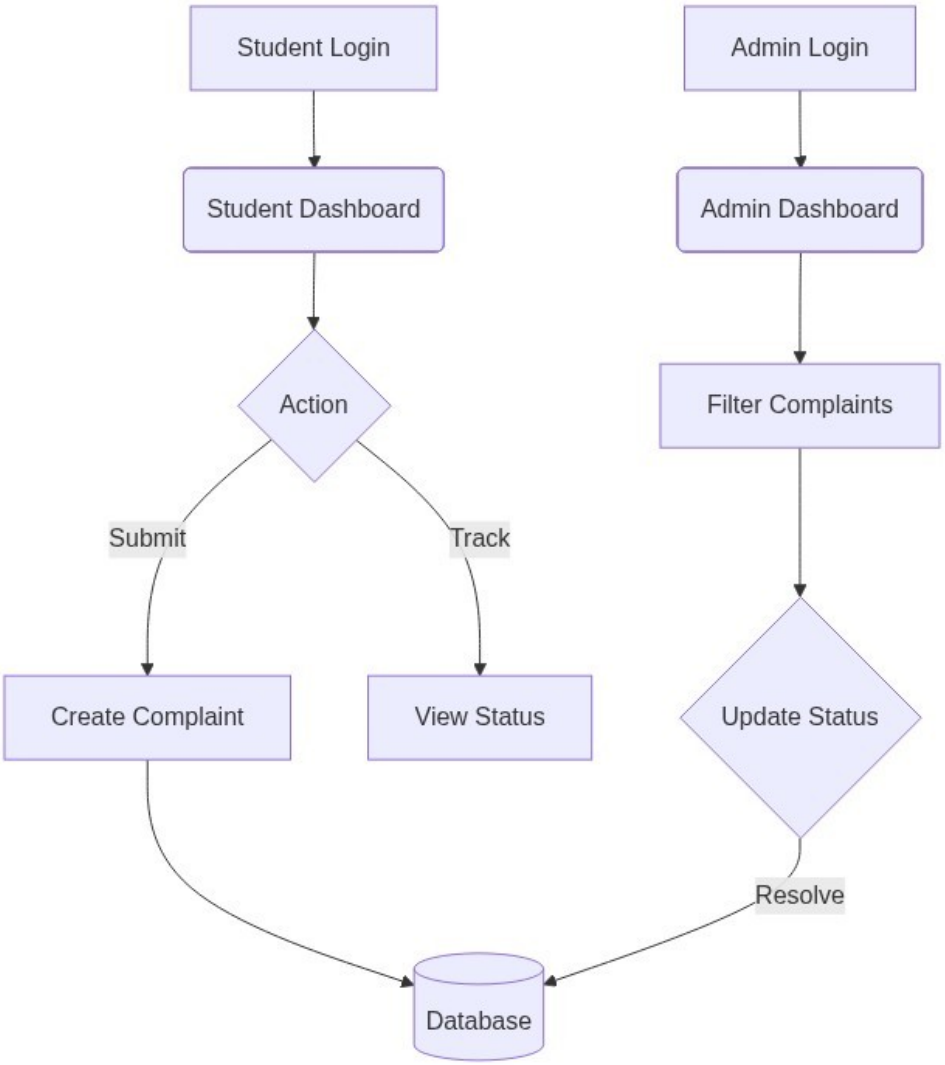

Figure 4.1: System Flowchart

5. Technology Stack

The project utilizes the MERN stack for its robust, asynchronous handling capabilities and modularity.

- Frontend: React.js (built with Vite) – Provides a fast, dynamic, Single-Page Application (SPA) experience with client-side routing.
- Styling: Vanilla CSS3 – Used for custom, premium visual aesthetics without heavy framework overhead.
- Backend Server: Node.js with Express.js – Acts as the REST API middleware, processing requests, authenticating tokens, and applying business logic constraints.
- Database: MongoDB with Mongoose ODM – Cloud-hosted NoSQL data layer ensuring high availability and schema validation.
- Security: Bcrypt.js (Password Hashing) and JSON Web Tokens (JWT) for stateless session management.

6. Project Structure

The system architecture is decoupled into two primary workspaces: frontend and backend.

Backend Architecture (Server)

- Controllers: Handle request/response logic for Authentication, Complaints, and Admin actions.
- Models: Define Mongoose Schemas (User Schema, Complaint Schema) ensuring database integrity.
- Routes: Expose RESTful endpoints (e.g., /api/auth, /api/complaints).
- Middleware: Intercepts requests to verify JWT authorization headers before granting access to controllers.

Frontend Architecture (Client)

- Components: Reusable UI blocks like Navigation, Complaint Forms, and Modal Dialogs.
- Pages: Major routing views (LandingPage, StudentDashboard, AdminDashboard, ProfileManager).
- Context/Hooks: Manage local application state and handle API dispatch operations using fetch/axios.

7. Development Environment

The following tools and environment configurations were utilized by the team to develop the CCMS:

- IDE: Visual Studio Code (VS Code) with ESLint and Prettier plugins for standardized code formatting.
- Runtime Environment: Node.js (v18+) & npm (Node Package Manager) for dependency management.
- Version Control System: Git, integrated with GitHub for collaborative branching, code reviews, and remote repository hosting.
- Database Management: MongoDB Atlas (Cloud) paired with MongoDB Compass (Local GUI) for direct data inspection and query testing.
- API Testing: Postman was used for endpoint verification and payload mocking before integrating the React client.

8. Project Management

The project was executed using Agile methodologies, segmented into three distinct one-week sprints. Tasks were tracked systematically.

Sprint Timeline

- Sprint 1 (Week 1): Frontend Setup. Included setting up React/Vite, designing UI screens, implementing client-side routing, and mocking dummy data.
- Sprint 2 (Week 2): Backend Setup. Included defining MongoDB schemas, building Express APIs, implementing JWT authentication, and conducting Postman testing.
- Sprint 3 (Week 3): Integration & Testing. Focused on connecting the React frontend to the Express backend via Axios, applying state safety rules, testing roles, and final bug fixing.

Team Structure

The workload was distributed across 5 team members to ensure modular feature ownership:

- Member 1: Authentication Module (Auth routes, bcrypt, login/register UI)
- Member 2: Complaint Submission Module (Form validation, database insertion)
- Member 3: Complaint Tracking Module (Student dashboard feeds, status history)
- Member 4: Admin Management Module (Filtering logic, status update API)
- Member 5 (Lead): Dashboard Analytics, GitHub merging, and full-stack integration.

9. Conclusion

The Campus Complaint Management System successfully bridges the gap between students and university administration. By migrating the grievance process to a MERN full-stack application, the project demonstrates how technical innovation can resolve critical real-world inefficiencies. The system ensures robust data security through stateless JWT tokens and enforces strict role-based access control, preventing unauthorized data manipulation.

Building this project served as a comprehensive learning opportunity in full-stack development, highlighting the complexities of API design, asynchronous state management, and schema architecture. Ultimately, the system establishes a highly transparent accountability loop, allowing students to trace their concerns and empowering staff to resolve issues systematically.

10. Appendix

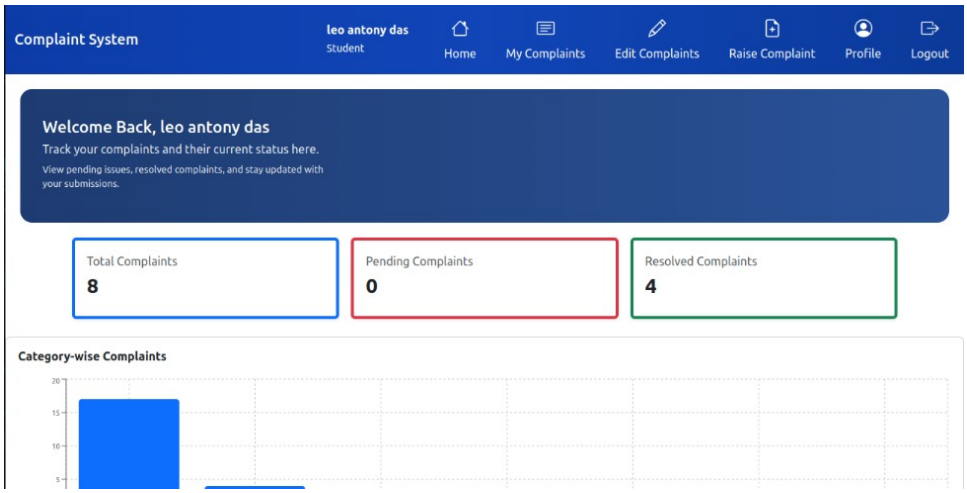

Figure 10.1: Student Dashboard Interface

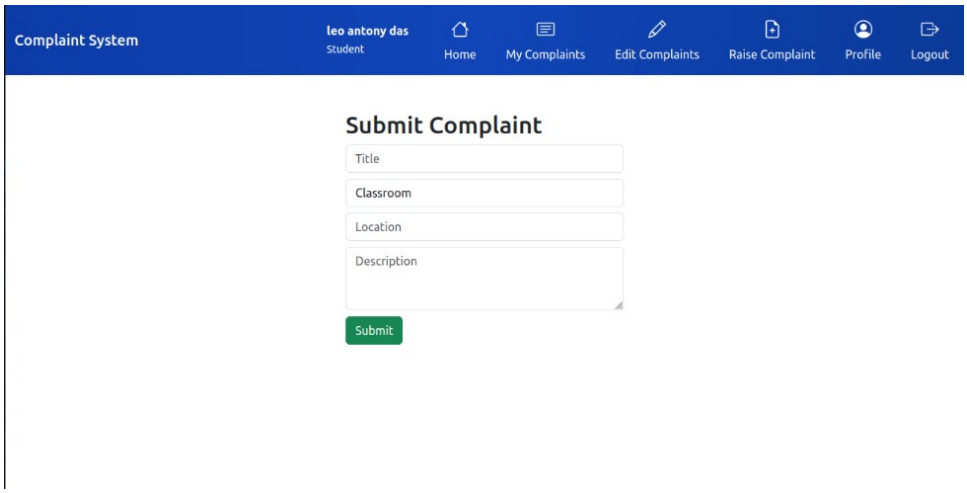

Figure 10.2: Submit Complaint Form

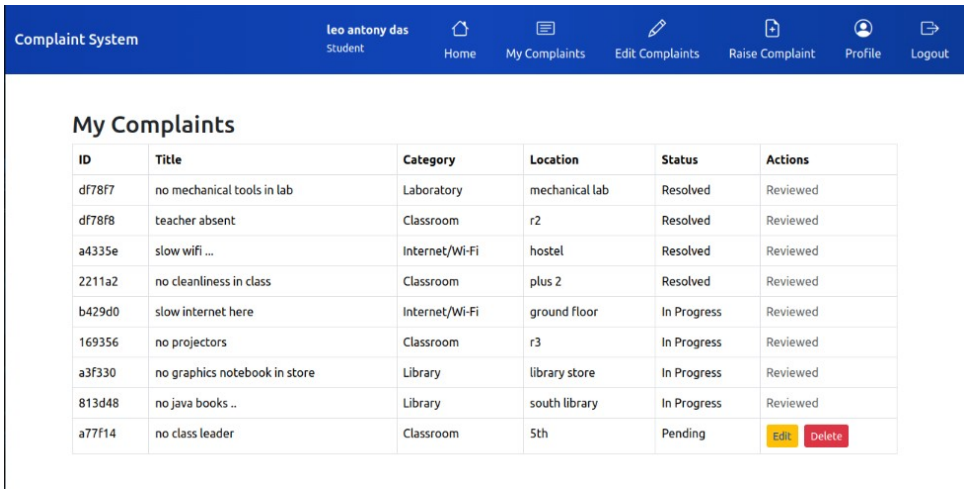

Figure 10.3: Personal Complaints Ledger

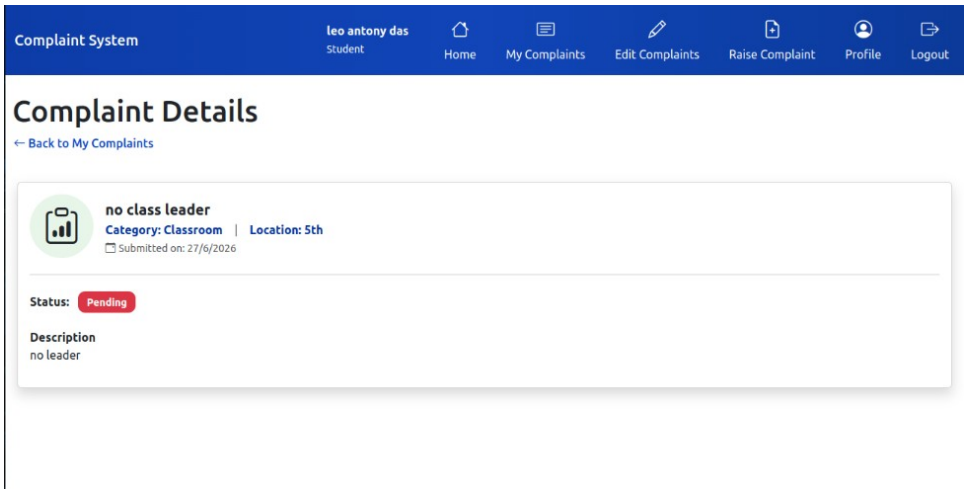

Figure 10.4: Detailed Complaint status view

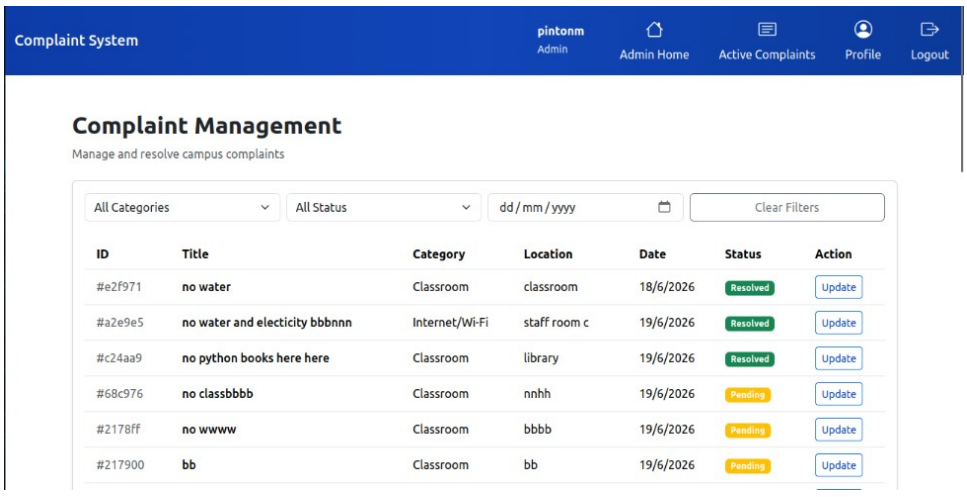

Figure 10.5: Admin Control Panel

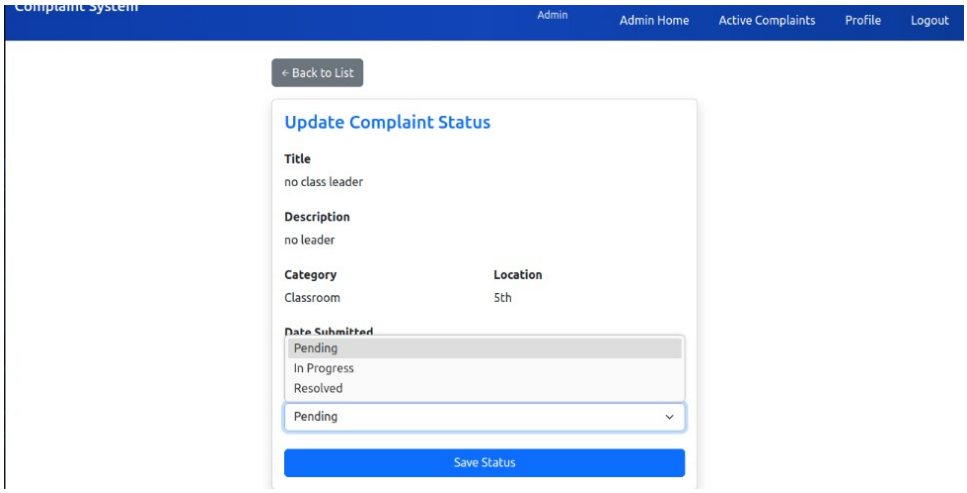

Figure 10.6: Admin Update Complaint interface

Login Credentials

Student Role:

- Email: leo000@gmail.com
- Password: leo001

Administrator Role:

- Email: pintonsebastian18@gmail.com
- Password: Pinto@8129

Links

Repo Link: https://github.com/mysterio05/Complaint_System